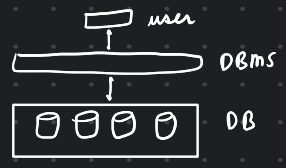
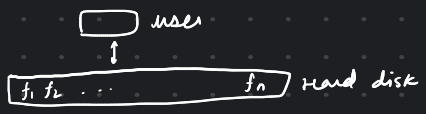
॥ श्रीगणेशाय नमः ॥

Databases

Database: collection of related data.
Database management system: software used to manage and access database files in a more efficient way.



Before DBMS, there were flat files in the OS.
Flat file system (OS files):
- DB files are managed by users without DBMS software.



The flat file system is good for small DBs, like in kbs or mbs.
If the DB is huge, then the flat file system fails.

Limitations of flat system vs adv of DBMS

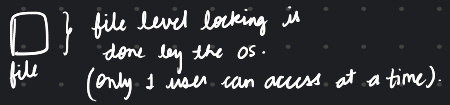
- too complex to manage application programs to access data. (the user needs to write programs for writing, deleting, searching data)

- DBMS supports data independency.
(The user can access data using an SQL interface without knowing storage information of DB files).
The user need not know how data is physically stored in the DB.

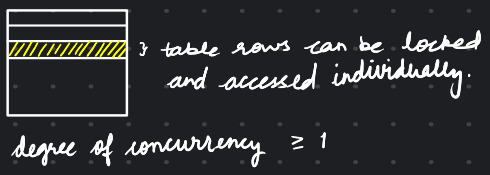
- more i/o cost to access data from DB files.

- because of indexing, i/o cost to access data will be lesser.

- degree of concurrency is very less.



- degree of concurrency is very high.



- too complex to maintain different levels of access control.

e.g.

student :

sid	sname	semail	saddress
✓	✓		

Say we want a teacher to only be able to see sid and sname.
Complex to implement this

- due to views (virtual tables), it's easy to manage access control.

- too complex to remove redundant data.

Sid	Student_Name	cid	course_name
1	A	2	C
1	A	3	D
1	A	4	E

- normalization of data makes it easy to maintain non-redundant data.

Integrity constraints

↳ correctness

We'll discuss constraints based on RDBMS (tables). It is a widely used data model, proposed by Codd.
Codd proposed 12 rules for design of DBMS software. These rules are called RDBMS guidelines.

Codd rule #1:
Data in a DB file should be in a tabular format.
(collection of rows and columns)
Any 2 rows of a DB file must not be the same. Each row is called a 'record' or 'tuple'.
The set of all rows of a file is called a relational instance or snapshot or record set.

attribute

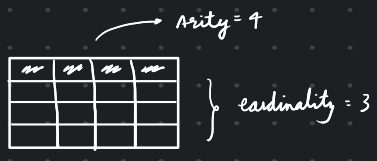
Sid	Sname	fees	DOB
S_1	A	100	1990
S_2	B	200	1990

} record

Record set or relational instance

Relational schema:
It is the definition/structure of a DB table.
(Informally, headings of the table columns). E.g. Student (Sid, Sname, marks).

Arity:
of fields of a DB table.



Cardinality:
of records of a DB table.

***
Candidate key:
No 2 records of an RDBMS table should be the same. So we rely on candidate keys.
A candidate key is the minimal set of attributes which can differentiate the records uniquely.

A	B	C
a	b	c
a	d	c
a	b	e

→ If (A, B) is the candidate key, 2 rows cannot have the same values for A and B.

AB is minimal. It means A & B cannot be candidate keys independently.

E.g.
Student(Sid, Sname, DOB)
Candidate key: Sid (the candidate key is underlined)
If Sid is the CK, then no 2 records can have the same Sid.

Q.

A	B	C
5	4	8
5	4	9
5	6	8
5	6	9
6	4	8

CK?
→ (A, B, C)

Q. Employee(Eid, Ename, DOB, Passport, Acc#, IFSC, Pan). What are the possible CKs?

→ Eid
Passport
Acc#, IFSC,
PAN
} Based on domain knowledge.

PRIMARY KEYS → NO NULL, UNIQUE

CANDIDATE KEYS → UNIQUE

SUPER KEYS → UNIQUE

Primary key:
It is any candidate key whose value is not NULL.
Atmost 1 primary key should be present for any RDBMS table.

NULL: unknown or non-existent.

Alternate keys:
All the candidate keys of a schema except the primary key are called alternate keys.

Declaring primary keys:

```
CREATE TABLE table_name (
    column1 datatype PRIMARY KEY,
    column2 datatype,
    ...
);
```

e.g.

```
CREATE TABLE employee (
    eid INT PRIMARY KEY,
    ename VARCHAR(100) NOT NULL,
    dob DATE,
    ppno VARCHAR(20) UNIQUE,
    pan_number VARCHAR(20) UNIQUE NOT NULL,
    acc_number VARCHAR(30),
    ifsc VARCHAR(20),
    UNIQUE (acc_number, ifsc)
);
```

Simple candidate key: A CK with only 1 attribute.
Composite candidate key: A CK with at least 2 attributes. E.g. (acc_number, ifsc)

Prime attribute:
An attribute that belongs to some candidate key of a relational schema. NOT PRIMARY KEY

CK = {A, B, C, D}
A is one of the prime attributes.

Non-prime attribute:
An attribute that doesn't belong to any of the candidate keys.

Eg. R(A, B, C, D, E)
CK = {AB, CD}
Primes: {A, B, C, D}
non-primes: {E}

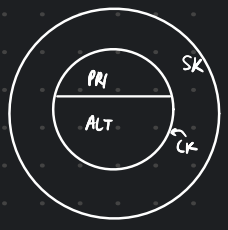
Super key:
Set of attributes that can differentiate records uniquely. It may or may not be a minimal attribute set.

If (A, B, C) is a super key, (A, B, C, D) is necessarily a super key.
(A, B) may be a super key.

```
CREATE TABLE example_table (
    column1 datatype,
    column2 datatype,
    column3 datatype,
    CONSTRAINT super_key_constraint UNIQUE (column1, column2)
);
```

E.g. Student (sid, sname, dob)
If candidate key = sid, superkeys are {(sid), (sid, sname), (sid, dob), (sid, sname, dob)}
of super keys: sid _ _ = 4 (2 · 2)
(sname: present/absent; dob: present/absent)

Any superset of a candidate key is a super key.
The candidate key is the minimal super key.
The primary key is one of the candidate keys.
The alternate keys are the candidate keys other than the primary.

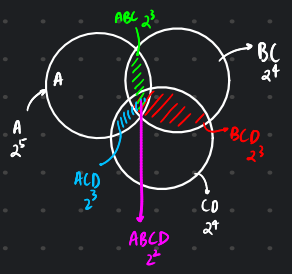


Q. R(ABCD). CK = {A}. # of super keys?
→ A _ _ _ = 2 · 2 · 2 = 8

Q. R(A,B,C,D,E). CK = (A, BC). # of super keys?
→
A _ _ _ _ = 2^4
BC _ _ _ = 2^3
common ABC _ _ = 2^2
total = $2^4 + 2^3 - 2^2 = 16 + 8 - 4 = 20$

Q. # of super keys with $R(A,B,C,D,E,F)$, $CK = \{A, BC, CD\}$.

→



$2^5 + 2^4 + 2^4 - (2^3 + 2^3 + 2^3) + 2^2$

$= 44$

Q. How many SKs with $R(A_1, A_2, \ldots, A_n)$ with $\{A_1A_2, A_2A_3\}$ as CKs?

→ A_1A_2 _ _ _ _ ... $= 2^{n-2}$

A_2A_3 ... $= 2^{n-2}$

$A_1A_2A_3$ _ _ _ ... $= 2^{n-3}$

$2^{n-2} + 2^{n-2} - 2^{n-3} = 2^{n-3}(2+2-1) = 3 \cdot 2^{n-3}$

Q. $R(A_1, A_2, \ldots, A_n)$. How many super keys are possible?

→ Max possible super keys will be when all records are unique for all attributes.

2^n possible subsets, ϕ cannot be a super key.

$2^n - 1$ possible super keys.

Foreign key:
Set of attributes in a table that refers to the primary key of another table, linking these two tables.

Student Table

sid (referenced attribute)	sname	smarks
1	John	85
2	Emily	92
3	David	78
4	Sarah	88

Course Table

cid (referenced attribute)	cname	institute
101	Math	ABC Institute
102	Science	XYZ College
103	Literature	PQR Academy
104	History	LMN School

Enroll Table

(sid: foreign keys, referencing attribute)

sid	cid
1	101
1	102
2	101
3	103
4	104

An sid would be in the enroll table (referencing) only if there's a corresponding sid in the student table (referenced).

An attribute could be referenced in the same table.
Consider an example: Employee(Eid, Ename, Supervisor_id).
A supervisor is also an employee. So a sid should be one of the eids.

eid	ename	sid
1	~	4
2	~	1
3	~	4
4		

Foreign key:
- used to create relationships between tables.
- defined over two relations (sometimes just one relation), referenced and referencing.

A foreign key is a set of attributes that references the primary key or alternate keys of the same or some other relation.



X, Y can be of the same/diff tables.

e.g.

Student:

sid	sname	dob
S1	A	1990
S2	A	1992
S3	B	1990
S4	B	1990
S5	A	1990

Enroll:

sid	cid	fees
S_1	C_1	500
S_1	C_2	100
S_2	C_3	100
S_4	C_4	100
S10	C_4	400

} not allowed. → foreign key constraint

```
CREATE TABLE child_table (
    ...
    FOREIGN KEY (child_column(s))
        REFERENCES parent_table (parent_column(s))
);

-- example
CREATE TABLE enrolled (
    student_id INT,
    course_id INT,
    grade CHAR(1),
    FOREIGN KEY (student_id) REFERENCES students (student_id),
    FOREIGN KEY (course_id) REFERENCES courses (course_id)
);
```

OR

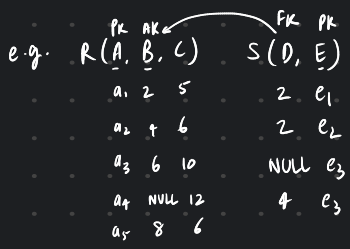
```
ALTER TABLE table_name
ADD CONSTRAINT constraint_name
FOREIGN KEY (column1, column2, ... column_n)
REFERENCES parent_table (column1, column2, ... column_n);
```

e.g. Employee

Eid	Ename	Supid
e_1	A	NULL
e_2	B	e_1
e_3	B	e_2
e_4	A	NULL
e_6	A	e_{10}

→ Foreign key attributes could be NULL

→ NOT ALLOWED cannot be something out of the referenced attribute values.



A foreign key can refer to alternate keys. FK attribute sets allow NULL attributes.
Referencing relations' records whose FK values are NULL are not related to any referenced relation record.

Between the referenced and the referencing relations, there's a 1:many relationship.
(Consider the example: 1 student enrolled in multiple courses).

What do we do when a referenced relation record is deleted?

a. on delete, no action
   Don't delete record.

b. on delete, cascade
   On deletion of referenced relation record,
   we are forced to delete all related referencing relation records.

department_id	department_name	manager_id
1	Sales	101
2	Marketing	102
3	HR	103
4	IT	104

employee_id	employee_name	department_id	salary
201	John	1	50000
202	Emma	1	55000
203	David	2	60000
204	Sarah	3	48000
205	Michael	3	52000
206	Lisa	4	65000

c. on delete, set NULL
   On deletion of referenced relation record,
   replace all the referencing relation records with NULL.

When a referenced relation record is updated?

a. on update, no action
   Don't update record.

b. on update, cascade
   On updation of referenced relation record,
   we are forced to update all related referencing relation records.

c. on update, set NULL
   On updation of referenced relation record,
   replace all the referencing relation records with NULL.

Normalization

Database anomalies

1. Introduction to database anomalies
2. Functional dependencies
3. Attribute closure, membership test
4. Finding candidate keys
5. Lossless joins and dependency
6. Normal forms
7. Finding highest normal form of relational schema
8. Decomposition into higher normal forms
9. Multi-valued dependencies and 4NF x
10. Canonical cover of FD set

It is used to eliminate redundancy in DB tables.
If 2/more independent relations are stored in a single relation, then it forms redundancies.

sid	sname	dob	cid	cname	instructor	fees
s1	a	1990	c1	DB	Korth	100
s2	a	1991	c1	DB	Korth	100
s3	b	1992	c1	DB	Korth	100
s3	b	1992	c2	algo	Cormen	200
s3	b	1992	c3	OS	Galvin	300

→ redundancy

↳ redundancy

Problems due to redundancy are database anomalies.

Insertion anomaly:
In order to insert a course in the above example, we need to insert values for sid, sname, dob (student data) as well. That makes no sense. Similarly, to insert a student, we need to insert a course.

Deletion anomaly:
If we delete the student S3, we'd lose course information about c2 and c3 as well.

Updation anomaly:
If one row is updated, the other rows fail to update, there will be inconsistencies.

sid	sname	dob	cid	cname	instructor	fees
s1	a	1990	c1	DB	Korth	100
s2	a	1991	c1	DB	Korth	100
s3	b	~~1992~~ 1993	c1	DB	Korth	100
s3	b	1992 ?	c2	algo	Cormen	200
s3	b	1992 ?	c3	OS	Galvin	300

All these anomalies can be avoided if there's no redundancy in the DB tables.
To reduce redundancy, a relation R is decomposed into R1, R2, ..., Rn.
This is called a normalized DB.

The table in the previous example can be split into 3 tables.

Student Table (sid, sname, dob):

sid (PK)	sname	dob
s1	a	1990
s2	a	1991
s3	b	1992

Course Table (cid, cname, instructor, fees):

cid (PK)	cname	instructor	fees
c1	DB	Korth	100
c2	algo	Cormen	200
c3	OS	Galvin	300

Enrolled Table (sid, cid):

sid (FK)	cid (FK)
s1	c1
s2	c1
s3	c1
s3	c2
s3	c3

Functional dependencies

Let X, Y be some sets of attributes of a relation R and t1, t2 be any tuples of the relation R.

X → Y is a FD in relation R if
if t1·X = t2·X then t1·Y = t2·Y

X	Y
x1	y5
x1	y5
x2	y2
x3	y2
x4	y4
x2	y2
x1	y5

Whenever X values are same,
the corresponding values of Y must be the same.
So X → Y

Here Y → X is not true.

Consider the example:

sid	sname	cid
s1	A	C1
s1	A	C2
s1	A	C3
s2	B	C4
s2	B	C1
s3	C	C2
s4	C	C3

Is this valid:

sid → sname ✓
sname → sid ×
cid → sid ×

Functional dependencies → trivial
non-trivial
semi non-trivial

Trivial FDs:
A FD X → Y is trivial if X ⊇ Y. It gives no new information.
E.g.
sid → sid,
(sid, sname) → sname

Non-trivial FDs:
A FD X → Y is non-trivial if X ∩ Y = φ. It gives some additional information.
E.g.
sid → sname,
(sid, cid) → fees

Semi non-trivial FDs:
A FD X → Y is non-trivial if X ∩ Y = φ. It gives some additional information.
E.g.
sid → (sid, sname),
Here sid → sid is trivial, sid → sname is non-trivial. Their combination, sid → (sid, sname) is semi non-trivial.
(sid, cid) → (cid, sname)
Here (sid, cid) → (cid) is trivial, (sid, cid) → sname is non trivial.

If X → YZ, then X → Y and X → Z.

Armstrong rules over FDs

Let X, Y, Z be some attributes over R, then

i. Reflexivity: (Trivial FD)
   (X → X) is always in R

ii. Transitivity:
   If (X → Y) and (Y → Z),
   then (X → Z).

iii. Augmentation:
   If (X → Y) is in R,
   then (XZ → YZ) is in R.

iv. Split rule:
   If (X → YZ),
   then (X → Y) and (X → Z).

NOTE: XY → Z cannot be split into X → Z and Y → Z.

v. Merge union:
   If (X → Y) and (X → Z),
   then (X → YZ).

Attribute closure

X is some set of relation R.
Then X+ = set of all attributes which can be determined by X.
   = set of all Y for which X → Y.

E.g. FD set = { A→B, C→D, AB→E, BE→C, EF→G }.

$A^+ = \{A, B, E, C, D\}$ $F^+ = \{F\}$

$AF^+ = \{A, B, E, C, D, F, G\}$

As AF^+ contains all attributes of R, AF is a super key of R.
AF is a candidate key as A^+ and F^+ are not super keys.

$(BE)^+ = \{B, E, C, D\}$

Super key:
X is some attribute set of relational schema R.
X is a super key of R if the closure of X, X^+, contains all the attributes of R.
(X determines all the attributes of R).

E.g. R(A, B, C, D, E, F) {AB→C, C→D, D→E, E→F, F→B }

$AB^+ = \{A, B, C, D, E, F\}$ ⇒ AB is a superkey. Also, any superset of the superkey is a superkey. (ABC, ABF... are also superkeys).

Candidate key (minimal superkey):
X is a candidate key of relation R if:
   - X is a super key of R
     (X^+ contains all attributes of R)
and - no proper subset of X is a super key of R.
     (∀ Y ⊂ X, Y^+ doesn't contain all attributes of R)

E.g. AB is a candidate key of R if
   - {A, B} is a super key of R
   - A and B are not super keys of R.

Q. R(A, B, C, D, E). {AB→C, C→D, B→E }. CK = ?

→ Method:
Go to RHS of the FDs and find the missing attributes. Those definitely ought to be in a CK.

{A, B} are missing.
$AB^+ = \{A, B, C, D, E\}$. ⇒ AB is a candidate key.
$A^+ = \{A\}$
$B^+ = \{B, E\}$

Q. Find all candidate keys of R with FD set F.

NOTE: Finding all candidate keys is an NP-hard problem.

$R(A, B, C, D)$

$F = \{AB \to C, C \to D, D \to B\}$

→

① Check if all attributes are present in the RHS of functional dependencies.

A B C D

x ✓ ✓ ✓

So every key should contain A. Check if that's a super key.

$A^+ = \{A\}$ ⇒ Not a super key. Now check 2-attribute keys.

$AB^+ = \{A, B, C, D\}$ ⇒ super key — A^+ and B^+ do not contain all attributes. ∴ AB is a candidate key.

$AC^+ = \{A, C, D, B\}$ ⇒ super key ⇒ candidate key

$AD^+ = \{A, D, B, C\}$ ⇒ super key ⇒ candidate key.

AB — B is prime. $D \to B$. Try replacing B by D. Check AD.

↓

AD — $C \to D$

↓

AC

If there's a non-trivial FD $X \to Y$ with Y as prime attribute, then R has at least 2 candidate keys. Try replacing Y with X to get a key.

AB, AC, AD are candidate keys.

Any superset will be a super key.

Q. $R(A, B, C, D, E)$. $FD = \{AB \to C, BC \to D\}$ candidate keys?

→ A, B, E not in RHS. Any key should have A, B, E.

$ABE^+ = \{A, B, E, C, D\}$ ⇒ ABE is the only candidate key as any super key will have A, B, E and that won't be CK.

↳ $A^+ = \{A\}$, $B^+ = \{B\}$, $E^+ = \{E\}$,

$AB^+ = \{A, B, C, D\}$, $BE^+ = \{B, E\}$, $AE^+ = \{A, E\}$

Q. $R(A, B, C, D, E, F)$ $FD = \{AB \to C, C \to D, CD \to BE, DE \to F, EF \to A\}$

→ All attributes on RHS.

$A^+ = \{A\}$, $B^+ = \{B\}$, $C^+ = \{C, D, B, E, F, A\}$, $D^+ = \{D\}$, $E^+ = \{E\}$, $F^+ = \{F\}$

⇓

C is a candidate key.

$AB^+ = \{A, B, C, D, E, F\}$ ⇒ candidate key.

no need to check those with C as C is a super key.

$AD^+ = \{A, D\}$ $AE^+ = \{A, E\}$.

Till now AB and C are candidate keys.

$EF \to A$

$EFB^+ = \{E, F, A, B, C, D\}$ $EF^+ = \{E, F, A\}$, $EB^+ = \{E, B\}$, $FB^+ = \{F, B\}$ ⇒ EFB is a candidate key.

$DE \to F$ and $CD \to BE$

EFB — CDF is not CK. (C is one)

↓

$DEB^+ = \{D, E, B, F, A, C\}$ $BD^+ = \{B, D\}$, $DE^+ = \{D, E, F, A\}$ ⇒ CK

candidate keys = $\{C, AB, BEF, BDE\}$

Q. $R(A, B, C, D, E, F)$ $FD = \{AB \to C, C \to DE, E \to F, F \to B\}$

→ A not in the RHS.

$A^+ = \{A\}$ ×

$AB^+ = \{A, B, C, D, E, F\}$ ⇒ CK

$F \to B$

$AF^+ = \{A, F, B, C, D, E\}$ ⇒ CK

$E \to F$

$AE^+ = \{A, E, F, B, C, D\}$ ⇒ CK

$C \to DE$

$AC^+ = \{A, C, D, E, F, B\}$ ⇒ CK

candidate keys = $\{AB, AC, AE, AF\}$

Q. $R(A, B, C, D, E)$. $FD = \{A \to B,\ B \to C,\ C \to D,\ D \to A\}$

→ E not on RHS. $E^+ = \{E\}$

$EA^+ = \{A, B, C, D, E\} \Rightarrow CK$

$EB^+ = \{B, C, D, A, E\} \Rightarrow CK$

$EC^+ = \{C, D, A, B, E\} \Rightarrow CK$

$ED^+ = \{D, A, B, C, E\} \Rightarrow CK$

$CK = \{AE, BE, CE, DE\}$

Q. $R(A, B, C, D, E, F)$. $FD = \{AB \to C,\ C \to A,\ BC \to D,\ ACD \to B,\ BE \to C,\ EC \to FA,\ CF \to BD,\ D \to E\}$

→ All attributes on RHS.

Ans:

$\{AB, CB, CD, BD, CF, EC, BE\}$

Membership test

X → Y is a member of FD set F iff X+ determines Y in F.

Q. FD = { AB → C, C → D, D → E, E → F, F → B }.
Which of the following are members of FD?

a) AB → F → AB^+ = {A, B, C, D, E, F} yes.
b) AC → B → AC^+ = {A, C, D, E, F, B} yes.
c) BC → A → A not on any RHS. No.

Equality of FDs

Let F and G be 2 FD sets. They are logically equal iff
1) F covers G (every FD in G must be a member of F, F ⊇ G) and
2) G covers F (every FD in F must be a member of G, G ⊇ F).

F⊇G and G⊇F ⇒ F≈G

Q. F = { A → B, B → C, C → A }
G = { A → BC, B → AC, BC → A, AB → C }

Which is true?
a) F ⊂ G ×
b) F ⊃ G ✓
c) F ≈ G ×

→ To check if F covers G:
F = { A → B, B → C, C → A }
G = { A → BC, B → AC, BC → A, AB → C }

A → BC : $A^+_{(F)}$ = {A, B, C} A → BC ∈ F
B → AC B^+ = {B, C, A} ∈ F
BC → A BC^+ = {B, C, A} ∈ F
AB → C AB^+ = {A, B, C} ∈ F

∴ All FDs in G are members of F.
⇒ F covers G.
F ⊇ G.

To check if G covers F:
A → B A^+ = {A, B, C} ∈ G
B → C B^+ = {B, A, C} ∈ G
C → A C^+ = {C} ∉ G

⇒ G does not cover F.
G ⊉ F.

Redundancy in relations can be eliminated by decomposing the relations.

1. Lossless join decomposition
2. Dependency preserving decomposition

Lossless join decomposition
A relational schema R with instance r is decomposed into sub-relations R1, R2, ..., Rn.
In general, R1 ⋈ R2 ⋈ ... ⋈ Rn ⊇ R.

⋈ → join

If R1 ⋈ R2 ⋈ ... ⋈ Rn ⊃ R, then it is lossy decomposition.
If R1 ⋈ R2 ⋈ ... ⋈ Rn ≡ R, then it is lossless decomposition.

Whenever we break a table into subtables, there should be attributes in common

---

R

sid	sname	cid
S_1	A	C_1
S_1	A	C_2
S_2	B	C_2
S_3	D	C_3

{sid → sname}
⇒ CK = (sid, cid)

sid	sname
S_1	A
S_2	B
S_3	D

{sid → sname}
CK = (sid)
↓
PK

sid	cid
S_1	C_1
S_1	C_2
S_2	C_2
S_3	C_3

NO FD.
CK = (sid, cid)

join ⇒

sid	sname	cid
S_1	A	C_1
S_1	A	C_2
S_2	B	C_2
S_3	D	C_3

Joining the decomposed tables gives the original table.
∴ It is a lossless join decomposition.

The common attributes are a primary key of one of the tables.
(sid)
∴ Lossless

---

sid	sname	cid
s1	A	C1
s1	A	C2
s2	B	C2
s3	B	C3

Sid	sname
S1	A
S2	B
S3	D

sname	cid
A	C1
A	C2
B	C2
B	C3

on joining

sid	sname	cid
S_1	A	C_1
S_1	A	C_2
S_2	B	C_2
S_2	D	C_3
S_3	B	C_2
S_3	B	C_3

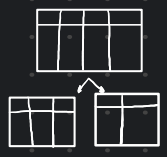
→ spurious tuples

The common attribute is not a primary key for one of the tables.
(sname)
So the decomposition is lossy.

A relational schema R with FD set F is divided into sub-relations R1, R2. The decomposition is lossless if

① $R_1 \cup R_2 = R$
② $R_1 \cap R_2 \rightarrow R_1$ (super key)
OR
$R_1 \cap R_2 \rightarrow R_2$

If relational R with record set is given, then use natural join method to determine if the decomposition is lossless.



If relational R is given with FD set, use

① $R_1 \cup R_2 = R$

② $R_1 \cap R_2 \rightarrow R_1$

OR

$R_1 \cap R_2 \rightarrow R_2$

Q. $R(A,B,C,D,E)$ FD set = $\{AB \rightarrow C, C \rightarrow D, B \rightarrow E\}$ Decomposition : $\{ABC, CD\}$

→ $ABC \cup CD = ABCD \neq ABCDE$ lossy.

Q. $R(A,B,C,D,E)$. FD: $\{AB \rightarrow C, C \rightarrow D, B \rightarrow E\}$. Decomposⁿ : (ABC, DE)

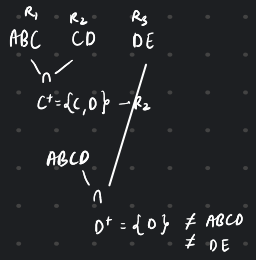
→ $R_1 \cup R_2 = R$ fine.
$R_1 \cap R_2 = \phi$ lossy.

Q. $R(A,B,C,D,E)$. FD = $\{AB \rightarrow C, C \rightarrow D, B \rightarrow E\}$. Decomposⁿ : $\{ABC, CDE\}$

→ $ABC \cup CDE = ABCDE$ ✓
$ABC \cap CDE = C$
$C^+ = \{C, D\}$ $C \not\rightarrow R_1$ $C \not\rightarrow R_2$
$C^+ \neq ABC$ $C^+ \neq CDE$.

Q. $R(A,B,C,D,E)$. FD = $\{AB \rightarrow C, C \rightarrow D, B \rightarrow E\}$. decomposition : $\{R_1(ABC), R_2(C,D), R_3(D,E)\}$

→ $R_1 \cup R_2 \cup R_3 = ABCDE$ ✓
check $R_1 \bowtie R_2$
$R_1 \cap R_2 = C$
$C^+ = \{C, D\} \rightarrow R_2$



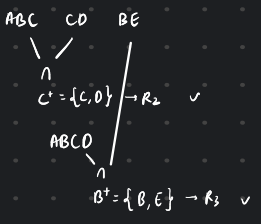
∴ lossless.

Q. $R(A,B,C,D,E)$. FD = $\{AB \rightarrow C, C \rightarrow D, B \rightarrow E\}$ Decomposⁿ : $\{R(AB), R_2(CD), R_3(BC)\}$

→ $R_1 \cup R_2 \cup R_3 = ABCD \neq ABCDE$ lossy.

Q. $R(A,B,C,D,E)$ FD = $\{AB \rightarrow C, C \rightarrow D, B \rightarrow E\}$ Decomposition : $\{ABC, CD, BE\}$

→ $R_1 \cup R_2 \cup R_3 = ABCDE$ ✓

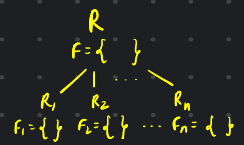


lossless

Dependency preserving decomposition

A relational schema R with FD set F is decomposed into sub-relations R1, ..., Rn with FD sets F1, ..., Fn.
In general, F1 ∪ ... ∪ Fn ⊆ F

If F1 ∪ ... ∪ Fn ≃ F, then it is a dependency preserving decomposition.
If F1 ∪ ... ∪ Fn ⊂ F, then it is not a dependency preserving decomposition.



If $F_1 \cup F_2 \cup \dots \cup F_n = F$,
all dependencies are preserved.

Q. $R(A, B, C, D, E)$. $F = \{A \to B, B \to C, C \to D, D \to E\}$. Decomposition: $\{AB, BC, CD, DE\}$

→ $R_1(AB)$:

We need to find FDs on A and B. They'll be of the form $A \to \dots$, $B \to \dots$, $AB \to \dots$ } AB is the table itself so ignore

$A^+ = \{A, B, C, D, E\}$

We'll take $A \to B$ as the table is AB. $A \to A$ is trivial.

$B^+ = \{B, C, D, E\}$

$B \to B$ is trivial.

$F_1 = \{A \to B\}$ for $R_1(AB)$

$R_2(BC)$:

$B^+ = \{B, C, D, E\}$ $B \to C$

$C^+ = \{C, D, B, E\}$ $C \to B$

$F_2 = \{B \to C, C \to B\}$

$R_3(CD)$:

$C^+ = \{C, D, B, E\}$ $C \to D$

$D^+ = \{D, B, E, C\}$ $D \to C$

$F_3 = \{C \to D, D \to C\}$

$R_4(DE)$:

$D^+ = \{D, B, E, C\}$ $D \to E$

$E^+ = \{E\}$

$F_4 = \{D \to E\}$

$F_1 \cup F_2 \cup F_3 \cup F_4 = G_7 = \{A \to B, B \to C, C \to B, C \to D, D \to C, D \to E\}$ $G_7 = F$

(present in F; $C^+ = \{C, D, B, E\}$; $D^+ = \{D, B, E, C\}$)

The decomposition is dependency preserving.

Q. $R(A, B, C, D)$. $F = \{AB \to CD, D \to A\}$, Decomposition: $\{ABC, BCD, AD\}$

→ For $R_1(ABC)$:

$A^+ = \{A\}$

$B^+ = \{B\}$

$C^+ = \{C\}$

$AB^+ = \{C, D\}$

$BC^+ = \{B, C\}$

$AC^+ = \{A, C\}$

$R_2(BCD)$

$D^+ = \{D, A\}$

$BD^+ = \{B, D, A, C\}$

$R_3(AD)$

$A^+ = A$

$D^+ = DA$

$F_1 = \{AB \to C\}$ $F_2 = \{BD \to C\}$ $F_3 = \{D \to A\}$

$F_1 \cup F_2 \cup F_3 = \{AB \to C, BD \to C, D \to A\} \neq F$ as $F_1 \cup F_2 \cup F_3$ doesn't have $AB \to CD$.

Not dependency preserving.

Normal forms

These are used to identify the degree of redundancy in a relation.
Redundancy could be due to:

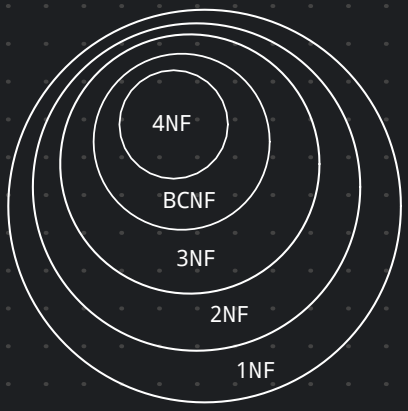
- non-trivial FDs

  $x \rightarrow y$

  (single-valued dependency)

- multi-valued dependencies.

1NF, 2NF, 3NF, and BCNF are normal forms that deal with single-valued dependencies.
BCNF completely eliminates redundancy through single-valued dependencies. Redundancy may exist over multi-valued dependencies.
4NF removes redundancy over single- and multi-valued dependences (no redundancy).



A relation in 4NF is in BCNF, 3NF, 2NF, and 1NF.
A relation in BCNF is in 3NF, 2NF, and 1NF. It may or may not be in 4NF.

1NF

A relation R is in 1NF if there are no multi-valued attributes in R.
Every attribute of R must be atomic/single-valued.
The default NF of RDBMS is in 1NF.

sid	sname	cid
s_1	A	c_1, c_2
s_2	A	c_2, c_3
s_3	B	c_3

not in 1NF

⇒

sid	sname	cid
s_1	A	c_1
s_1	A	c_2
s_2	A	c_2
s_2	A	c_3
s_3	B	c_3

} 2
} 2

1NF

sid	sname	cid	phone	email
s1	A	c1,c2	p1,p2,p3	e1,e2,e3,e4
s2	B	c2,c3	p4,p5	e4,e5

Q. How many rows in 1NF?
→ $2 \times 3 \times 4 = 24$ rows
$2 \times 2 \times 2 = 8$ rows.
32 rows

Q. If FD = {sid → sname}. CK = ?
→ {(sid, cid, phone, email)}
LHS of FD is not superkey. ⇒ redundancy. (sid is a partial key).
sid → sname } partial dependency.
↳ not a superkey.

When do we get redundancy?

$X \to Y$ is an FD of R. It gives redundancy if
- $X \to Y$ is a non-trivial FD and
- X is not the superkey of R.

$X \to Y$ does not form redundancy iff:
- $X \supseteq Y$ (non-trivial) or
- X is a superkey of R.

- When LHS is not a super key:
  R(sid, sname, cid). FD = {sid → sname}
  ⇒ CK = (sid, cid)

sid → sname
↳ part of CK is determining non-prime attribute.
↳ partial dependency.

sid	sname	cid
s1	A	c1
s1	A	c2

↳ unnecessary repetition.

} here a student can enroll in many courses.
Repeating sname would be unnecessary.
But sid → sname makes us keep them together.

To remove that redundancy, we'd split the table. R_1(sid, sname), R_2(sid, cid)

e.g. R(eid, ename, DOB, rating, salary). FD = {(eid → ename, dob, rating) (rating → salary)}
CK = (eid).
(rating → salary) would cause redundancy as rating is not a super-key.

e.g. R

sid	sname	dob
s1		
s2		
s3		
s4		

{(sid → sname, dob)}
⇒ CK = (sid).
∴ LHS of FD is super key. ⇒ No redundancy.

Causes of redundancy

R(A, B, C, D, E, F)
FD = {
AB → C
B → D
D → E
AE → F
C → A
}
CK = {AB, CB}

These would cause redundancy.

Prime attributes = {A, B, C}
non-primes = {D, E, F}

① B → D
↳ non-prime
↳ proper subset of CK (partial key)
{ partial key → non-prime } redundancy.

② D → E
↳ non-prime
non-prime
{ non-prime → non-prime } redundancy

③ AE → F
↳ non-prime
prime
non-prime
proper subset of CK
{ prime + non prime → non prime } redundancy
proper subset of CK

④ C → A
↳ proper subset of CK
proper subset of CK
{ proper subset of CK → proper subset of CK } redundancy.

2NF

A relation R is in 2NF if there are no partial dependencies in R.



X is a CK. Z is non-prime. $Y \to Z$ is called a partial dependency.
$Y \subset X$

NOTE: If {A, B} are primes, $A \to B$ is not a partial dependency. partial key → non-prime is a partial dependency } not allowed in 2NF.

3NF

A relation R is in 3NF if every non-trivial FD X→Y in R is such that
- X must be a super key,
- or Y must be a prime attribute.

$X \to Y$
↳ superkey and/or prime

e.g. CK = {AB, CD}

$AB \to E$ } 3NF
↳ SK

$D \to B$ } 3NF
↳ prime

BCNF: Boyce Codd Normal Form

It does not allow any kind of redundancy.
A relation R is in BCNF if every non-trivial FD X→Y in R has X as the super key.

$X \to Y$
↳ super key

2NF: No partial dependency. (subset of key → non-prime) } (proper subset of CK)$^+$ → (only primes)

3NF: $X \to Y$
(= super key OR = prime)

BCNF: $X \to Y$
= super key

Test in the order: BCNF, 3NF, 2NF, 1NF

check if $X \to Y$
= super key → BCNF

check if $X \to Y$
↳ prime → 3NF

Q. $R(A, B, C, D, E)$ $FD = \{ABD \to C, BC \to D, CD \to E\}$. What is the highest NF of R?

→ CK: $AB^+ = \{AB\}$
$ABD^+ = \{ABCDE\}$ ✓
$ABC^+ = \{ABCDE\}$ ✓

CK = {ABD, ABC}
Primes = {A, B, C, D} non-primes = {E}

$ABD \to C$ — key — BCNF

$BC \to D$ — partial key → prime — 3NF

$CD \to E$ — not in 3NF

To check for 2NF:
CK = {ABD, ABC}
$A^+ = A$ $B^+ = B$ $D^+ = D$
$AB^+ = AB$ $BD^+ = BD$ $AD^+ = AD$
$C^+ = C$
$BC^+ = BCDE$ → non-prime
partial key → non-prime
⇒ NA in 2NF

⇒ 1 NF

Q. $R(A, B, C, D)$. $FD = \{AB \to C, BC \to D\}$
→ CK: $AB^+ = \{ABCD\}$ ✓
CK = {AB} non-primes = {C, D}

$AB \to C$ — key — BCNF ✓

$BC \to D$ — BC: non-prime; D: non-prime
BCNF ×
3NF ×
To check for 2NF: CK = {AB} → proper subsets = {A, B}
$A^+ = A$
$B^+ = B$
⇒ No partial dependencies.
⇒ 2NF.

2NF.

Q. $R(A, B, C, D)$ $FD = \{AB \to C, C \to A, AC \to D\}$
→ CK: $B^+ = B$
$AB^+ = ABCD$ ✓
$BC^+ = BCAD$ ✓
CK = {AB, BC} non-primes = {D}

$AB \to C$ — key — BCNF ✓

$C \to A$ — C: prime; A: prime
BCNF ×
3NF ✓ (RHS prime)

$AC \to D$ — AC: primes; D: non-prime
BCNF ×
3NF ×
2NF: CK = {AB, BC}
$A^+ = A$ $C^+ = AD$ → non-prime
$B^+ = B$
⇒ partial dep ⇒ 2NF ×

1NF

Q. $R(ABCDEF)$ $FD = \{AB \to C, C \to DE, E \to F, F \to B\}$
→ CK: $AB^+ = ABCDEF$ ✓
$AF^+ = AFBCDE$ ✓
$AE^+ = AEFBCD$ ✓
$AC^+ = ACDEFB$ ✓
CK = {AB, AF, AE, AC} non-primes = {D}

$AB \to C$ — key — BCNF ✓

$C \to DE$ → $C \to D$ (partial key, non-prime); $C \to E$
3NF ×
2NF ×

Decompose $X \to YZ$ into $X \to Y$, $X \to Z$ to test for 3NF.

1NF

Q. $R(A, B, C, D, E, F)$ · $FD = \{AB \to C,\ C \to D,\ CD \to AE,\ DE \to F,\ EF \to B\}$

→ CK: $C^+ = \{CDAEFB\}$

AB
AEF
ADE

$CK = \{C, AB, AEF, ADE\}$ non-primes = φ

3NF

As all attributes are prime attributes, the table is surely in 3NF.

$AB \to C$	$C \to D$	$CD \to AE$	$DE \to F$	$EF \to B$
↓ key	↓ key	↓ key	↓ NA key	↓ not key
			↓ prime	↓ prime
BCNF ✓	BCNF ✓	BCNF ✓	BCNF ✗	BCNF ✗
			3NF ✓	3NF ✓

Q. $R(A, B, C, D, E)$ $\{A \to B,\ B \to C,\ C \to D,\ D \to A\}$

→ CK: $AE^+ = \{A, E, B, C, D\}$

DE
CE
BE

$CK = \{AE, BE, CE, DE\}$

$A \to B$	$B \to C$	$C \to D$	$D \to A$
↓ not key	↓ prime	↓ prime	
BCNF ✗			
3NF ✓	3NF ✓	3NF ✓	3NF ✓

All RHS are prime attributes.

Q. $R(A, B, C, D)$ · $\{A \to D,\ B \to AC,\ C \to D\}$

→ CK: $A^+ = \{A, B, C, D\}$

B

$CK = \{A, B\}$

$A \to B$	$B \to AC$	$C \to D$
BCNF ✓	BCNF ✓	BCNF ✗
		3NF ✗

To check for 2NF: $\{A, B\}$

No partial dependency.
proper subset of key → non-prime ✗✗

There's no proper subsets for A and B, so no partial dependencies exist.
⇒ 2NF ✓

When all the candidate keys are simple keys (not composite), then the relation is surely in 2NF.

A relation schema R with no non-trivial FDs is always in BCNF.

If there are no FDs, all the attributes together will be a candidate key.
The table would be in BCNF, since there would be no redundancy.

How is redundancy formed?

When X is not a superkey in X → Y.

Q. A relational schema R with only 2 attributes is surely in _____.

→ For $R(AB)$, all the possible FD sets are

① $\{A \to B\}$ $CK = A$ $A \to B$ BCNF ✓

② $\{B \to A\}$ same

③ $\{A \to B, B \to A\}$ $CK = \{A, B\}$. BCNF ✓

④ $\{\} = \varphi$ BCNF ✓

Decomposition of relations into higher NFs

2NF decomposition

$R(A, B, \ldots, Y, Z)$ $\quad FD = \{ \cdots, Y \to Z \}$ ↳ partial

Say $Y^+ = \{Y, Z\}$

↓

$(\cdots Y)(YZ)$

$Y \to Z$ } In $R(Y, Z)$, Y is not a partial key. It is a key.

Consider R(sid, sname, dob, cid). F = { (sid → sname, dob) }.

$CK = \{(sid, cid)\}$

sid → sname, dob } partial dependency.
↳ partial key

$sid^+ = \{sid, sname, dob\}$
↳ make new table

There should be something in common.
↳ superkey of one of the tables.

We decompose into R(sid, sname, dob) and R(sid, cid).
(sid referenced, sid referencing)

F = {sid → sname, dob} BCNF ✓

F = { } BCNF ✓

Problems with partial dependencies:
(sid → sname, dob)

sid	sname	dob	cid
s1	A	1990	c1
s1	A	1990	c2
s1	A	1990	c3

redundancy

sid	sname	dob
s1	A	1990

sid	cid
s1	c1
s1	c2
s1	c3

Q. $R(A, B, C, D, E, F)$ $\quad \{AB \to C, C \to D, B \to E, E \to F\}$. Break into 2NF.

→ $CK = \{AB\}$

$B \to E$ ↳ partial key $\quad B^+ = \{B, E, F\}$

We'll split R(B, E, F).



→ 2NF.

2NF ✓

3NF ✗ due to E → F
R(B E F)

BE: B → E — BCNF ✓
EF: E → F — BCNF ✓

3NF ✗ due to C → D
R(A, B, C, D)

ABC: AB → C — BCNF ✓
CD: C → D — BCNF ✓

Here we got all tables dependency preserving and lossless join decomposition.

That may or may not happen always.

Take the problematic FDs and split the table.

Q. $R(A,B,C,D,E,F,G)$ $\{AB \to C,\ C \to D,\ A \to E,\ B \to F,\ F \to G\}$

→ CK = {AB}

$\{AB \to C,\ C \to D,\ A \to E,\ B \to F,\ F \to G\}$

partial

$A^+ = \{AE\}$ $D^+ = \{BFG\}$

ABCDEFG

AE CD BFG

R_1: AE, $A \to E$, BCNF ✓

R_2: ABCD, $AB \to C$, $C \to D$, 3NF ✗, $C^+ = \{CD\}$

R_3: BFG, $B \to F$, $F \to G$, 3NF ✗, $F^+ = \{FG\}$

AE ($A \to E$, BCNF ✓) — ABC ($AB \to C$, BCNF ✓) — CD ($C \to D$, BCNF ✓) — BF ($B \to F$, BCNF ✓) — FG ($F \to G$, BCNF ✓)

lossless ✓
dependency-preserving ✓
BCNF ✓

All FDs are preserved and we get a lossless join decomposition.

Q. $R(ABCDE)$. $FD = \{AB \to C,\ BC \to A,\ AC \to B\}$.

→ CK: $\{ABDE, BCDE, ACDE\}$ primes = $\{A,B,C,D,E\}$

$AB \to C$ $BC \to A$ $AC \to B$

all are prime attributes. ∴ It is in 3NF.

$AB \to C$ BCNF ✗ $BC \to A$ BCNF ✗ $AC \to B$ BCNF ✗

AB is partial key

Here $AB \to C$ ↳ But C is prime. So it is not a partial dependency.

$AB^+ = \{ABC\}$ $BC^+ = \{ABC\}$ $AC^+ = \{ABC\}$

ABCDE

R_1 (ABC)
$AB \to C$
$BC \to A$
$AC \to B$

R_2 (DE AB)
no non-trivial FDs.
⇒ BCNF ✓

we could've chosen DEAC or DEBC as well, depending on which key we choose for R_1.

CK = {AB, BC, AC}
⇒ BCNF ✓

Q. $R(A,B,C)$ $\{AB \to C,\ C \to A\}$

→ CK = {AB, BC}

$AB \to C$ — key — BCNF ✓

$C \to A$ — prime — 3NF ✓, BCNF ✗

$C^+ = \{C,A\}$

split CA

CA: $C \to A$, BCNF ✓

BC: no non-trivial dependencies, BCNF ✓

lossless join ✓
$AB \to C$ is not preserved.
dependency preserving ××
BCNF ✓

Here if we want to preserve dependencies, we'd have to stay in 3NF.

Q. R(ABCD) {ABC→D, D→B}

→ CK = {ABC, ADC}

all are prime attributes. ⇒ 3NF.

ADC→D — BCNF ✓

D→B — BCNF ✗

D⁺ = {D, B}

R(DB) — D→C

R(ACD) — no non-trivial FD

ABC→D is not preserved.

dependency preserving ✗

lossless join ✓

BCNF ✓

Database design goals based on normalization	1NF	2NF	3NF	BCNF	4NF
① lossless join decomposition	✓	✓	✓	✓	?
② dependency	✓	✓	✓	may/may not be possible	may/may not be possible
③ 0% redundancy	✗	✗	✗	✓ over single-valued FDs ✗ over multi-valued FDs	✓

The most accurate NF among these is 3NF.
Lossless joins, dependency preserving, and 0% redundancy is possible.

Database Design Goals Based on Normalization	1NF	2NF	3NF	BCNF	4NF
Lossless Join Decomposition	✔	✔	✔	✔	May or may not be achievable
Dependency Preservation	✔	✔	✔	May or may not be achievable	May or may not be achievable
Zero Redundancy	✗	✗	✗	Achieved over single-valued FDs	Achieved over both single- and multi-valued FDs

Minimal set or canonical cover

It is an FD set that contains no redundant FDs.

Procedure to find minimal cover

1. Split the functional dependencies such that their RHSs contain single attributes.

$AB \to CD \Rightarrow AB \to C$, $AB \to D$

2. Find the redundant FD and delete it from the set.

$\{A \to B, B \to C, A \to C\}$

↳ redundant. $A^+ = ABC$ with or without this FD.

3. Find the redundant attributes on the LHS and delete them.

If $AB \to C$ and A^+ contains B,

↳ we don't need this.

Q. $\{A \to C, AC \to D, E \to AD, E \to H\}$

→ step 1: only single attributes in RHS.

$\{A \to C, AC \to D, E \to A, E \to D, E \to H\}$

step 2:

① $A \to C$

check A^+ without this. $A^+ = \{A\}$

so we can't remove it.

② $AC \to D$ ×

$AC^+ = \{AC\}$

can't remove

③ $E \to A$ ×

$E^+ = \{EDH\}$ can't remove

④ $E \to D$ ×

$E^+ = \{EAHCD\}$

we can delete this. ←

⑤ $E \to H$ ×

$E^+ = \{EACD\}$

can't delete this.

So the FD set left is $\{A \to C, AC \to D, E \to A, E \to H\}$

step 3: $\{A \to C, AC \to D, E \to A, E \to H\}$

check the LHS for redundancy.

$A \to C$: LHS is not redundant.

$AC \to D$

Try removing A. $C^+ = \{C\}$ does not have A. So A is reqd.

Try removing C. $A^+ = \{ACD\}$. A already gives a C.

So if we have A, we can get D.

So we don't need $AC \to D$.

we are fine with $A \to D$ as $A \to C$ is implied.

$\{A \to C, A \to D, E \to A, E \to H\}$

Now these can be combined.

$\{A \to CD, E \to AH\}$.

we've reduced $F_1 = \{A \to C, AC \to D, E \to AD, E \to H\}$ to $F_2 = \{A \to CD, E \to AH\}$

$F_1 \geq F_2$ and $F_2 \geq F_1$. F_2 is the canonical cover of F_1.